

CS 3006

Parallel and Distributed Computing

Lecture 3

Danyal Farhat

FAST School of Computing

NUCES Lahore

Amdahl's Law and Karp-Flatt Metric

Lecture's Agenda

- **Amdahl's Law of Parallel Speedup**
- **Karp-Flatt Metric**
- **Gustafson's Law**
- **Types of Parallelism**
 - **Data Parallelism**
 - **Functional Parallelism**
 - **Pipelining**

Lecture's Agenda (Cont.)

- **Multi-Processor vs. Multi-Computer**
- **Cluster vs. Network of Workstations**
- **Summary**
- **Additional Resources**

Amdahl's Law

- Amdahl's law was **formulized** in 1967
- It shows an **upper-bound** on the **maximum speedup** that can be achieved
- Suppose you are going to design a **parallel algorithm** for a problem
- Further suppose that fraction of total time that the algorithm must consume in **serial** executions is 'F'
- This implies fraction of **parallel** portion is (1- F)
- Now, Amdahl's law **states** that

$$\text{Speedup}(p) = \frac{1}{F + \frac{1-F}{P}}$$

- Here 'p' is total number of available **processing** nodes

Amdahl's Law (Cont.)

Derivation:

- Let's suppose you have a sequential code for a problem that can be executed in total $T(s)$ time
- $T(p)$ be the parallel time for the same algorithm over p processors.
- Then speedup can be calculated using:

$$\text{Speedup}(p) = \frac{T(s)}{T(p)}$$

- $T(p)$ can be calculated as:

$$T(p) = \text{serial comput. time} + \text{Parallel comp. time}$$

$$T(p) = F \cdot T(s) + \frac{(1-F) \cdot T(s)}{P}$$

Amdahl's Law (Cont.)

Derivation:

- Again

$$\text{Speedup}(p) = \frac{T(s)}{T(p)} \Rightarrow \frac{T(s)}{F \cdot T(s) + \frac{(1-F) \cdot T(s)}{P}}$$

$$\Rightarrow \text{Speedup}(p) = \frac{1}{F + \frac{1-F}{P}}$$

- What if you have infinite number of processors?

Amdahl's Law (Cont.)

- **Example:** Suppose we have a system with one processor and a serial program. We want to upgrade this system and we have the following two options.
 - a) Increase the number of processors from 1 to 8 and parallelize 40% of the code.
 - b) Increase the number of processors from 1 to 4 and parallelize 70% of the code.
- Calculate speedup in each case and identify which one is better?

Karp-Flatt Metric

- Amdahl's law ignore the parallelization overhead due to which it **overestimate** the achievable speedup.
- The Karp - Flatt metric is used to calculate **serial fraction** for a given parallel configuration.
- Experimentally **determined** serial fraction e takes into account parallel overhead.
- Detects other **sources of overhead** or inefficiency ignored in speedup model
 - Process startup time
 - Process synchronization time
 - Imbalanced workload
 - Architectural overhead

Karp-Flatt Metric (Cont.)

- If a parallel program is exhibiting a **speedup S** while using **P processing units** then experimentally determined serial fraction e is given by :

$$e = \frac{1/S - 1/p}{1 - 1/p}$$

- Experimentally determined serial fraction e may either stay **constant** with respect to p (meaning that the parallelization overhead is **negligible**) or **increase** with respect to p (meaning that parallelization overhead **dominates** the speedup)

Karp-Flatt Metric (Cont.)

- **Proposed** by Alan H. Karp and Horace P. Flatt in 1990
- To **determine** to what extent an algorithm is parallelized
- If metric value remains **consistent** w.r.t. increasing number of processors and speedups, we need to reduce sequential fraction of the program
- If metric value **increases** as increasing number of processors and speedups, we need to reduce parallelize overheads

Karp-Flatt Metric – Example

- **Example 1:** Suppose in a parallel program, for the given processors, you gained a speedup given in second row, determine sequential fraction of your program.

Solution: Compute $e(n, p)$ corresponding to each data point:

p	2	3	4	5	6	7	8
S	1.82	2.50	3.08	3.57	4.00	4.38	4.71
$e(n, p)$	0.1	0.1	0.1	0.1	0.1	0.1	0.1

Since the experimentally determined serial fraction $e(n, p)$ is not increasing with p , the primary reason for the poor speedup is the 10% of the computation that is inherently sequential. Parallel overhead is not the reason for the poor speedup.

Karp-Flatt Metric – Example (Cont.)

- **Example 2:**

Benchmarking a parallel program on 1, 2, ..., 8 processors produces the following speedup results:

p	2	3	4	5	6	7	8
s	1.87	2.61	3.23	3.73	4.14	4.46	4.71

What is the primary reason for the parallel program achieving a speedup of 4.71 on 8 processors?

Solution:

p	2	3	4	5	6	7	8
s	1.87	2.61	3.23	3.73	4.14	4.46	4.71
e	0.07	0.075	0.08	0.085	0.09	0.095	0.1

Since the experimentally determined serial fraction e is steadily increasing with p , parallel overhead also contributes to the poor speedup.

Gustafson's Law

- John L. Gustafson and Edwin H. Barsis presented **Gustafson's law** in the article Reevaluating Amdahl's Law in 1988.
- Gustafson's law addresses the **shortcomings of Amdahl's law**, which is based on the assumption of a **fixed problem size**, that is of an execution workload that does not change with respect to the improvement of the resources.
- Gustafson's law instead proposes that programmers tend to **increase the size of problems** to fully exploit the computing power that becomes available as the resources improve.

Gustafson's Law (Cont.)

- Gustafson estimated the speedup S of a program gained by using parallel computing as follows:

$$\begin{aligned} S &= s + p \times N \\ &= s + (1 - s) \times N \end{aligned}$$

- S is the **theoretical speedup** of the program with parallelism
- N is the **number of processors**
- s and p are the fractions of time spent executing the **serial parts** and the **parallel parts** of the program
- Gustafson observed from their workloads that time for the **serial part typically does not grow** as the problem and the system scale, i.e. s is fixed.

Types of Parallelism

Data Parallelism

- When there are independent tasks applying the **same operation** to different elements of a data set
- Example code

```
for i=0 to 99 do
  a[ i ] = b[ i ] + c [ i ]
Endfor
```
- Here **same operation** addition is being performed on first 100 elements of 'b' and 'c' arrays
- All 100 iterations of the loop could be **executed simultaneously**

Types of Parallelism (Cont.)

Functional Parallelism

- When there are independent tasks applying **different operations** to different data elements
- Example code
 1. $a=2$
 2. $b=3$
 3. $m = (a+b) / 2$
 4. $s = (a \times a + b \times b) / 2$
 5. $v = s - m^2$
- Here third and fourth statements could be performed **concurrently**

Types of Parallelism (Cont.)

Pipelining

- Usually used for the problems where **single instance** of the problem can not be parallelized
- The output of **one stage** is input of the other stage
- Dividing whole computation of each instance into **multiple stages** provided that there are multiple instances of the problem

Types of Parallelism (Cont.)

Pipelining (Cont.)

- An effective method of **attaining parallelism** on the uniprocessor architectures
- Depends on **pipelining abilities** of the processor
 - Pentium 2 supports **12 stage** pipelining
 - Pentium 4 supports **20 stage** pipelining at 2.0 GHz

Types of Parallelism (Cont.)

- **Pipelining Example: Assembly line analogy**

Time	Engine	Doors	Wheels	Paint
5 min	Car 1			
10 min		Car 1		
15 min			Car 1	
20 min				Car 1
25 min	Car 2			
30 min		Car 2		
35 min			Car 2	
40 min				Car 2

Sequential Execution

Types of Parallelism (Cont.)

- **Pipelining Example: Assembly line analogy**

Time	Engine	Doors	Wheels	Paint
5 min	Car 1			
10 min	Car 2	Car 1		
15 min	Car 3	Car 2	Car 1	
20 min	Car 4	Car 3	Car 2	Car 1
25 min		Car 4	Car 3	Car 2
30 min			Car 4	Car 3
35 min				Car 4

Pipelining

Types of Parallelism (Cont.)

- **Pipelining Example: Assembly line analogy**

Cycles	Fetch	Decode	Execute	Save
1	Inst 1			
2	Inst 2	Inst 1		
3	Inst 3	Inst 2	Inst 1	
4	Inst 4	Inst 3	Inst 2	Inst 1
5		Inst 4	Inst 3	Inst 2
6			Inst 4	Inst 3
7				Inst 4

4-stage Pipelining

Multi-Processor vs. Multi-Computer

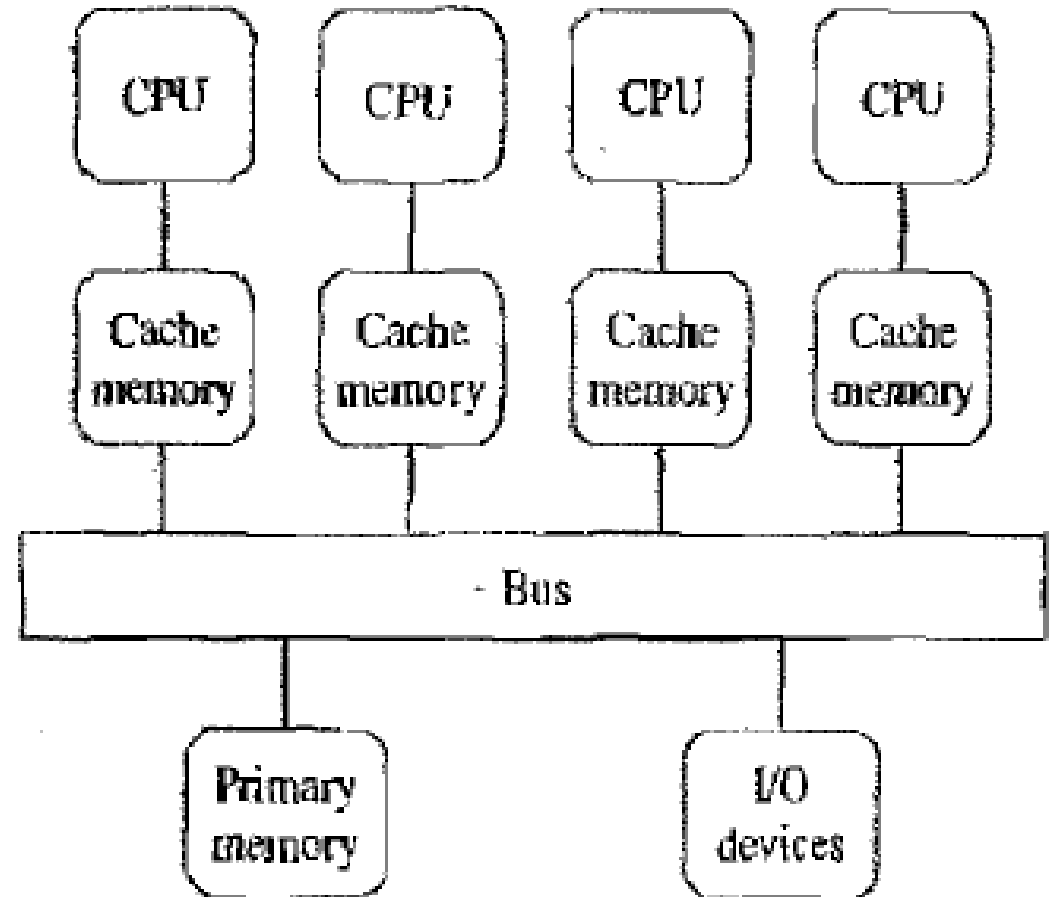
Multi-Processor

- Multiple CPUs with a shared memory
- Same address on two different CPUs refers to the same memory location
- Generally two categories
 - Centralized Multi-processor
 - Distributed Multi-processor

Multi-Processor vs. Multi-Computer (Cont.)

Centralized Multi-Processor

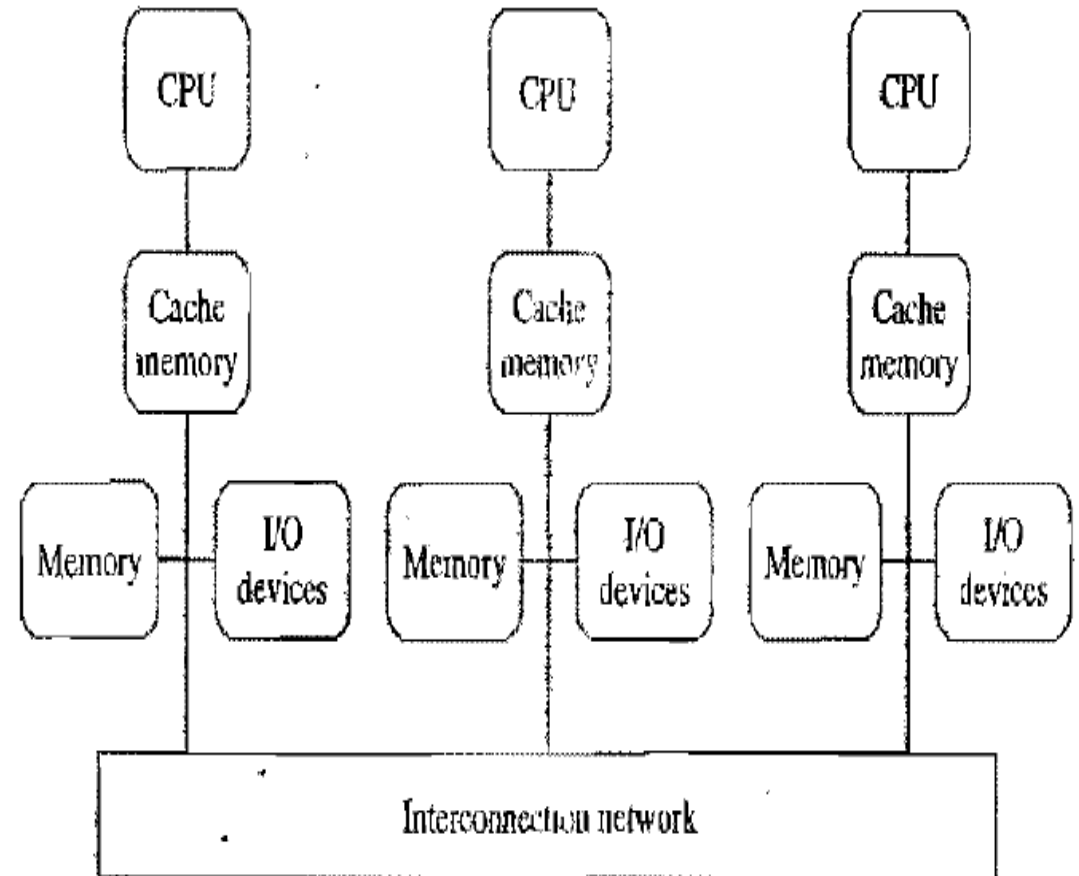
- Additional CPUs are **attached** to the system bus, and all the processors share the same primary memory
- All the memory is at **one place** and has the same access time from every processor
- Also known to as UMA (Uniform Memory Access) Multi-processor or SMP (Symmetrical Multi-processor)



Multi-Processor vs. Multi-Computer (Cont.)

Distributed Multi-processor

- Distributed collection of memories **forms** one logical address space
- Again, the same address on different processors **refers** to the same memory location
- Also known as Non-Uniform Memory Access (NUMA) architecture
- Because, memory access time **varies** significantly, depending on the physical location of the referenced address



Multi-Processor vs. Multi-Computer (Cont.)

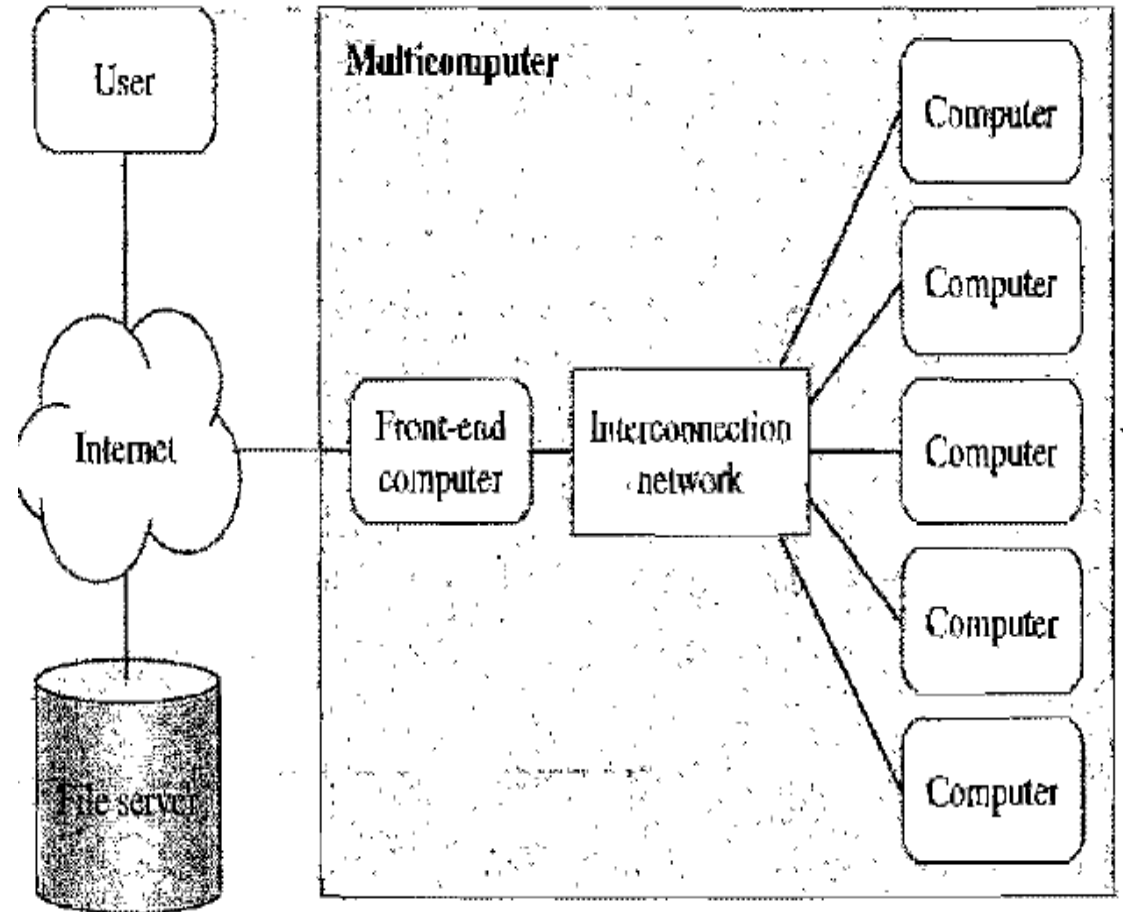
Multi-Computer

- Distributed memory, **multi-CPU** computer
- Unlike NUMA architecture, a multicomputer has **disjoint** local address spaces
- Each processor has **direct access** to their local memory only
- Same address on different processors **refers** to two different physical memory locations
- Processors **interact** with each other through passing messages

Multi-Processor vs. Multi-Computer (Cont.)

Asymmetric Multi-Computers

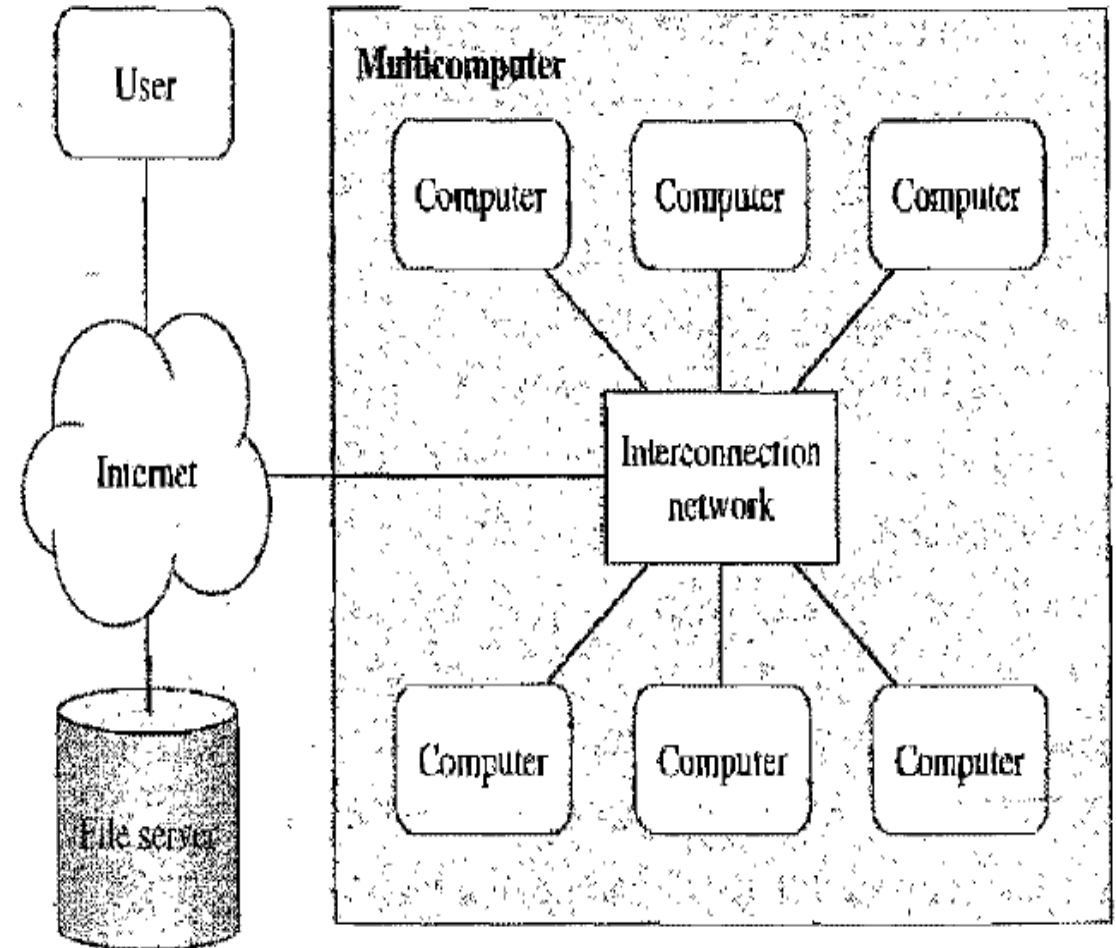
- A **front-end** computer that interacts with users and I/O devices
- The **back-end** processors are dedicatedly used for “number crunching”
- Front-end computer executes a full, multi-programmed OS and **provides** all functions needed for program development
- The backends are **reserved** for executing parallel programs



Multi-Processor vs. Multi-Computer (Cont.)

Symmetric Multi-Computers

- Every computer **executes** same OS
- Users may **log** into any of the computers
- This **enables** multiple users to concurrently login, edit and compile their programs
- All the nodes can **participate** in execution of a parallel program



Cluster vs. Network of Workstations

Cluster

- Usually a **co-located** collection of low-cost computers and switches, dedicated to running parallel jobs
- All computer run the **same version** of operating system
- Some of the computers may not have **interfaces** for the users to login
- Commodity cluster uses **high speed networks** for communication such as fast Ethernet at 100Mbps etc.

Network of workstations

- A **dispersed collection** of computers
- Individual workstations may have **different Operating systems** and executable programs
- Users have the power to **login and power off** their workstations
- Ethernet speed for this network is usually **slower**.
 - Typical in **range** of 10s Mbps

Cache Coherence

- When we are in a distributed environment, each CPU's cache needs to be consistent (continuously needs to be **updated** for current values), which is known as cache coherence

Snooping

- Snoopy protocols achieve **data consistency** between the cache memory and the shared memory through a bus-based memory system
- Write-invalidate and write-update **policies** are used for maintaining cache consistency

Snooping Protocol Types

Write-invalidate (mostly used)

- The processor that is writing data causes copies in the caches of all other processors in the system to be **rendered invalid** before it changes its local copy

Write-update

- The processor that is writing the data **broadcasts** the new data over the bus
- All caches that contain copies of the data are then **updated**

Branch Prediction

- Branch prediction is a technique used in CPU design that attempts to **guess the outcome** of a conditional operation and prepare for the most likely result
- Issues in pipelining of branch prediction:
 - If the **prediction is true** then the pipeline will not be flushed and no clock cycles will be lost
 - If the **prediction is false** then the pipeline is flushed and starts over with the current instruction

Summary

- **Amdahl's Law**

- Predict theoretical speedup when using improved resources

$$\text{Speedup} = \frac{1}{(1-p)+p/n}$$

- **p**: ratio of parallelizable code **n**: number of processors
- Scaling falls off as the number of processors increases due to synchronization issues and parallel overhead

Summary (Cont.)

- **Karp-Flatt Metric**

- Calculate serial fraction for a given parallel configuration
- Detects sources of overhead or inefficiency ignored in the speedup model
 - ✓ Process startup time, Process synchronization time
 - ✓ Imbalanced workload, Architectural overhead

$$e = \frac{1/s - 1/p}{1 - 1/p}$$

- e is constant with respect to p - parallelization overhead is negligible
- e increases with respect to p - parallelization overhead dominates the speedup

Summary (Cont.)

■ Types of Parallelism

➤ Data Parallelism

- ✓ Independent tasks applying the same operation to different elements of a data set

➤ Functional Parallelism

- ✓ Independent tasks applying different operations to different data elements

➤ Pipelining

- ✓ Effective method of attaining parallelism on the uniprocessor architectures

Summary (Cont.)

- **Multi-Processor vs. Multi-Computer**

- **Multi-Processor - Multiple CPUs with a shared memory**

- ✓ Centralized Multi-Processor - All the memory is at one place and has Uniform Memory Access (UMA) architecture
 - ✓ Distributed Multi-processor - Distributed collection of memories forms one logical address space. Use Non-Uniform Memory Access (NUMA) architecture

- **Multi-Computer - Each processor has disjoint memory**

- ✓ Asymmetric Multi-Computers
 - A front-end computer interacts with users and I/O devices
 - The back-end processors are dedicated for processing
 - ✓ Symmetric Multi-Computers
 - Every computer executes same OS
 - This enables multiple users to concurrently login, edit and compile their programs

Summary (Cont.)

- **Cluster vs. Network of Workstations**

- **Cluster** - Co-located collection of computer with same version of OS
- **Network of Workstations** - Dispersed collection of computers with different versions of OS

- **Cache Coherence**

- **Cache** continuously needs to be updated for current values

Summary (Cont.)

- **Snooping**

- Snoopy protocols achieve data consistency between cache and memory through a bus based memory system
- Write invalidate & write update policies are used

- **Branch Prediction**

- Used in CPU design to guess the outcome of a conditional operation and prepare for the most likely result

Additional Resources

- **Introduction to Parallel Computing** by Ananth Grama and Anshul Gupta
 - Chapter 2: Parallel Programming Platforms
 - Chapter 5. Analytical Modeling of Parallel Programs
 - ✓ 5.4 Scalability of Parallel Systems

Questions?

Class Activity Time